Quickstart — Claude Science with the Research Toolkit

Welcome. This is the two-minute version: just enough to be productive in your very first Claude Science session with the bundle installed. When you want the full story behind any point below, the **Detailed Guide** (`CS_USAGE_DETAILED`) expands all of it.

> The machine-optimized root, `CS_QUICKSTART.machine.md`, is authoritative; this is its human-readable twin.

What It Is

Claude Science is an agentic pair-scientist that lives in a **remote sandbox** — the browser is its only channel to you, so there is no local shell, no hooks, and no local audio. It works in **turns**: you set a goal, it plans and acts through the `host.*` API, and you review the result. This toolkit adds **52 skills, 18 agent profiles, and fail-closed kernel gates** to your account, so every session inherits the same methodology. (See "The Map" in the Detailed Guide.)

The Day-One List

The things to know on your first day:

1. **Start.** Open a conversation in your Science project under an installed **profile** — `GENERALIST` is the daily driver, with 17 named specialists alongside it. There is no launch command: your installed skills, profiles, and project-memory are already live in the account. (Installing them is covered in `CS_INSTALL_STARTER_v2.11.md`.)
2. **Memory is your context.** This replaces Claude Code's per-repo `CLAUDE.md` auto-load: there is no config file that loads each session. Instead, durable **project-memory** auto-recalls into context — which makes memory the *poison surface*, so keep it current, because stale memory is re-read as fact. **Artifacts** are inert until something searches for them. (See "Context.")
3. **Prompt.** Type an instruction and be specific — **name your output artifacts**. (See "The Loop.")
4. **The agency dial — three skills, not a keystroke.** Load `plan` for deliberation (map the territory and get your go/no-go before any scope-defining step), `collab` for the middle default (it surfaces the non-trivial calls without gating every step), and `solo` for full autonomy (run a handed-off task to completion, no check-ins). You invoke each by naming the skill. (See "The Loop" and "Part B — The Toolkit Overlay.")
5. **Profiles are the agents.** A profile is the Claude Science analogue of a Claude Code subagent: pick one for the session, and dispatch specialists as **children** through `host.delegate`. (See "Instructions — How You Steer It.")
6. **Models are routed at delegation, per child.** `host.delegate` routes each child by **tier** — Opus 4.8 (T1) and Fable 5 (the hardest tier) for hard problems, Sonnet 5 (T2/T3) for routine work, and Haiku 4.5 (T4) for trivial tasks. Never Claude Opus 5, and never the bare `opus` alias (it resolves to Opus 5) — use full IDs. (See "Instructions" and the `delegation-planning` skill.)
7. **Effort rides the tier** (T1 is max, down to T4 low) — nothing for you to preset. (See "Instructions.")

8. **Skills auto-load.** Just describe the task — fit a GAM, gap-fill, QC a series, build a brms model — and the right skill loads itself through `host.skills`; you can also load one by naming it. (See "Instructions.")
9. **Kernel sidecars.** A skill may carry a `kernel.py` of **plain callable functions** — no CLI, no `__main__` guard. Use it from a repl cell with `exec(host.skills.read("<skill>", "kernel.py")["content"])`, then call the function (for example, the `delegation-planning` routing gates). (See "Instructions" and "Automation.")
10. **Delegation and scale.** `host.delegate` children run **asynchronously** — you collect their reports, steer them, or stop them, and long work advances in the background while you keep going. (See "Delegation and Scale.")
11. **The disciplines do the worrying for you.** They ride in your profile, the skills, and the kernel gates — there is no always-on rules file on Claude Science. Claude verifies before saying "done," fixes root causes rather than symptoms, and holds the standing mandate: clarity, accuracy, traceability, reproducibility. You can rely on all of this as given. (See "Part B — The Toolkit Overlay.")
12. **Completion ping.** Claude Science has no turn-end hook, so the `audible-alert` skill fires an **agent-initiated** ping at turn boundaries — Claude beeps itself, as its last action. (See "Automation.")
13. **Handoff and resume.** There is no `/baton` and no file-drop on Claude Science. Instead, durable **project-memory** persists across sessions, and **saved artifacts carry lineage** — together they are your cold-resume trail, so persist the load-bearing facts before you stop. (See "Context.")
14. **Context** auto-summarizes as it fills; start a **new conversation** for an unrelated task. (See "Context.")

Your First Session (the recipe)

Open a Science conversation under a profile, load `plan`, and describe the task — data artifact, model, output artifact. Read and approve the plan Claude returns, then let it work, delegating specialists through `host.delegate` as needed. Review the results, persist the durable facts to project-memory and save your artifacts with lineage, and start a new conversation before the next task. (See "Your First Research Session, Step by Step.")

For More

Everything above expands in the Detailed Guide (`CS_USAGE_DETAILED`) — the full roster of fifty-two skills and eighteen profiles, the `host.*` API surface, memory-as-poison-surface, the delegation topologies, and extension authoring on Claude Science. The install mechanics and acceptance probes live in `CS_INSTALL_STARTER_v2.11.md`, at the bundle root.